1

# Fundamental Web API Patterns

Fundamental patterns and recommendations



Includes content adapted from *API Design Pattern*, 2021, Manning Publications && *RESTful Java Web Services, 3rd Ed.* , 2017, Packt Publishing

## Fundamental Patterns

1. Resource Identification
2. Standard Methods
3. Field Masks
4. **Custom Methods**
5. Long-running Operations
6. Re-runnable Jobs



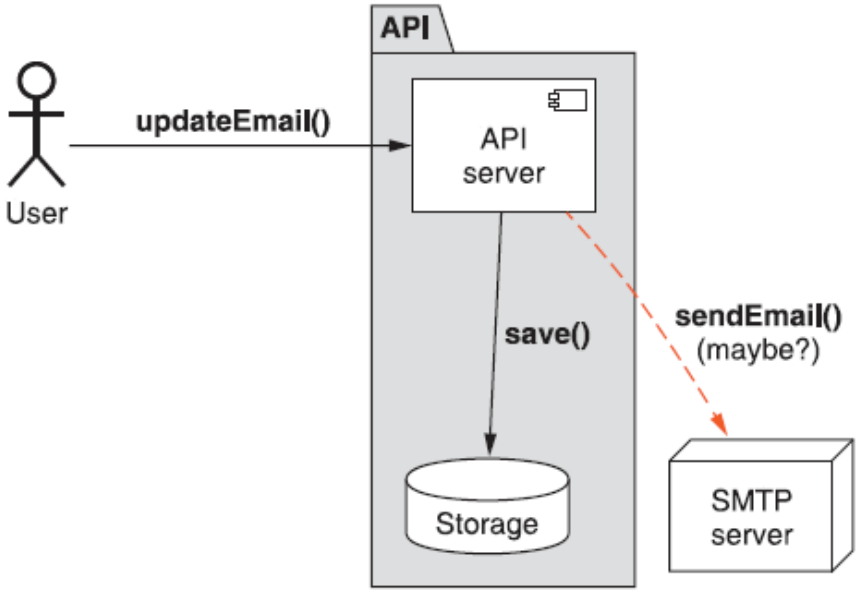
## Custom Methods

- Standard methods cannot cover all scenarios:
  - Some actions may not fit cleanly in standard methods
  - If standard methods were used anyway, it would result in a surprising, confusing and complex API.
- Just because Standard methods can do anything, it doesn't mean they should!
  - However, custom methods can safely be designated to do pretty much anything that is necessary.



# Custom Methods

- *Side-effects* in standard methods should be avoided. However, these are fine in custom methods.





## Custom Methods

- There are generally no hard and fast rules for custom methods out of the box.
- The choice is left to the API designer to decide on a set of rules and precedents,
- However, those rules should then applied be consistently across that API.



## Custom Methods

- Custom methods are implemented with the POST operation and a special ":" separator.
  - Other methods (e.g. GET, DELETE) can be used , but this is less common.

```
POST /mails/2123:send
```



# Custom Methods

- Custom methods are tailored to circumstances, so they may target collections or individual resources.
  - *As a general guideline, if a request handles multiple resources, the custom method should target the corresponding collection for consistency.*

Avoid

```
POST /user/1:exportMails
```

Do this instead

```
POST /user/1/emails:export
POST /users/-/emails:archive
POST /users/1:export
```



# Custom Methods

- Sometimes custom methods can be stateless (i.e. not attached to any resource)
- Stateless methods are usually implemented using a top-level resource.
  - *This resource can commonly (though not always) represent the API user (project, account, organization, etc), which might be necessary anyway for billing purposes.*

```
POST /text:translate

POST /projects/12:translate
```

## Fundamental Patterns

1. Resource Identification
2. Standard Methods
3. Field Masks
4. Custom Methods
5. **Long-running Operations**
6. Re-runnable Jobs



# Long Running Operations

- Sometimes, API calls cannot provide an immediate response. These are long-running operations (LRO).
- It might not be convenient for the API consumer to just wait for an LRO to conclude.
- The user might wish to:
  - Do other work while waiting
  - Be informed of the progress of LRO
  - Pause, resume or cancel the operation



## Long Running Operations

- It is not elegant, practical or convenient to rely on the same design for consistently fast and consistently slow operations.
  - *This makes it harder to debug and create responsive applications.*
  - *How long is long enough?*
- The LRO pattern offers a way to provide asynchronous operation to the API.
  - *Akin to Futures or Promises offered in standard programming languages.*

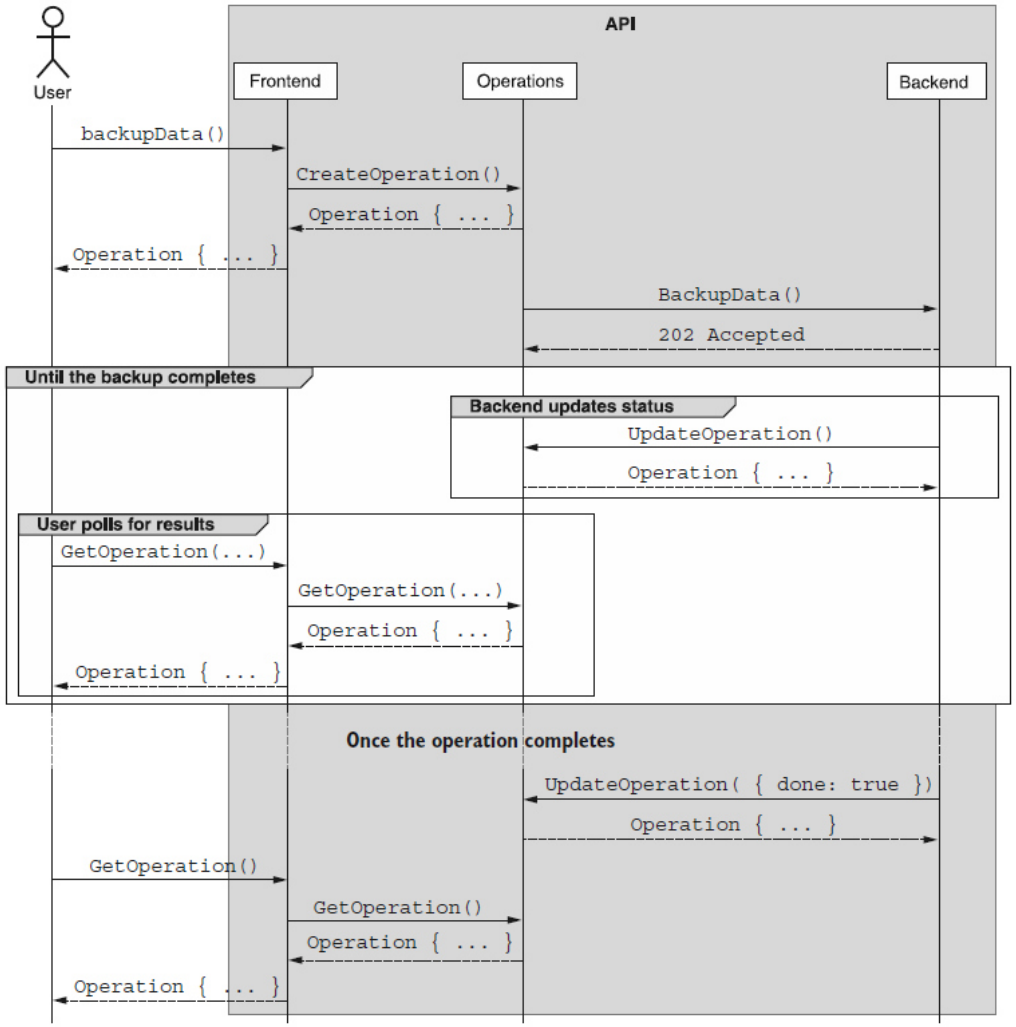


## Long Running Operations

- Rather than waiting for completion of the work ans then return the result, when executing a request for an LRO, the server will:
  1. Initiate the necessary work
  2. Immediately return a placeholder object – the LRO
- The placeholder object is a resource that will allow meta-information to be made available, as well as acting on the work being performed.

# Long Running Operations

- LROs act as an intermediate between the user and the back-end processes.
- They allow the user to inspect the status of the operation, and often also pausing, resuming or cancelling it.





# Long Running Operations

- LRO resources will typically support the standard GET and LIST methods.
  - `GET /operations/1`
  - `GET /operations`
- Often, they will also support custom methods for pausing, resuming and cancelling the operation.
  - `POST /operations/1:pause`
  - `POST /operations/1:resume`
  - `POST /operations/1:cancel`



# Long Running Operations

- A LRO resource will usually contain the following or similar fields:
  - **id** – The unique resource id
  - **done** – A Boolean indicating (not necessarily successful) job completion.
  - **result** – The result of the job. May be null if no result is returned. Might be information concerning error instead (code number, message text, details) .
  - **meta-data** – Meta data concerning the operation (completion percentage, start date/time, etc…)



# Long Running Operations

- Example:

```
{
  "id": 1,
  "done": true,
  "result": {
    "total": 1342
  },
  "metadata": {
      "type" : "data verification",
      "startTime": "2022-04-23T18:25:43",
      "endTime": "2022-04-23T19:05:33",
      "progress": 100
    }
}
```



## Long Running Operations

- Operation resource location:
  - It is far more convenient to have a centralized, top-level collection of LRO rather than having resource specific LRO collections, i.e.:

Rather than

`/chatRooms/1/operations/2`

it is more convenient to have

`/operations/1`



## Long Running Operations

- The client can retrieve the results of an LRO by:
  - Polling an LRO resource until it “done=true”
  - Using a custom operation when getting the LRO to indicate it wants to wait until completion (keeping the connection alive).

```
GET /operations/2:wait
```



# Long Running Operations

- Error Handling:
  - On event of an error, the error code should be returned as an error code in the LRO result.
  - Errors accessing the LRO itself should be handled using the HTTP response status

**This ensures it is abundantly clear to the client if the error was triggered during execution of the LRO job, or during retrieval of the LRO object itself.**



## Long Running Operations

- Cancelling jobs:

```
POST /operations/1:cancel
```

  - The cancel method should return the cancelled LRO (with done set to *true*). It should block until job is cancelled (i.e. cancel is not an LRO itself).
  - Ideally, all created and allocated resources must be destroyed. If that is not the case, provide information to the user so it can do the clean up himself.



## Long Running Operations

- Pausing/resuming jobs:

```
POST /operations/1:pause
POST /operations/1:resume
```

  - Use a “paused” field in metadata to indicate the operation has been paused (it is NOT “done”).
  - Pause and resume are not LRO themselves (i.e. they are synchronous operations).



## Long Running Operations

- Exploring Operations:

```
GET /operations/?done=false
```

  - Similar to a standard LIST method, but filtering is very useful to allow common operations such as identifying ongoing or paused operations to be possible.



## Long Running Operations

- For how long should operations be persisted?

- **Easy and simple answer:** Forever.
- **Rolling window:** Purge based on completion timestamp and set time window (e.g. 30 days). Metadata will include “expireTime”.

- ***Other approaches and options :*** Cascade delete with resources, archiving first and then deleting… Avoid all these unless really necessary.

## Fundamental Patterns

1. Resource Identification
2. Standard Methods
3. Field Masks
4. Custom Methods
5. Long-running Operations
6. **Re-runnable Jobs**

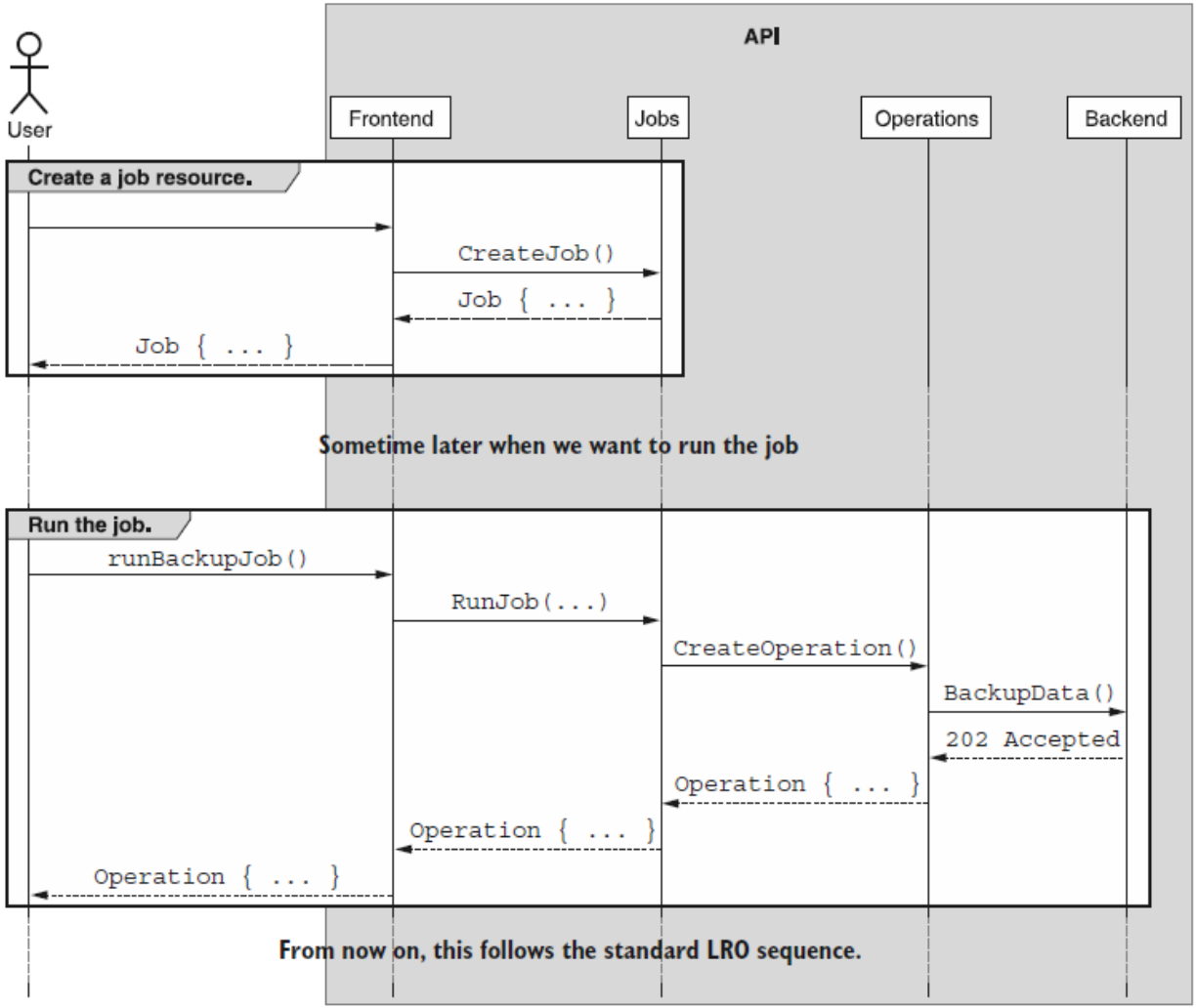


## Re-runnable Jobs

- Re-runnable jobs are resources that represent operations that are stored in the system and can be executed at a later date.
  - Allows some operations to be provided without requiring the user to specify all the details every time it is executed.
  - Allows API-side scheduled execution of operations
  - Allow implementing different policies for running and configuring a job (e.g. Development vs Operations)

# Re-runnable Jobs

- Job resources are defined in advance.
- Requests for execution are sent at a later date.





## Re-runnable Jobs

- Re-runnable job resources store the configuration necessary to execute the desired operation.

```
{
  "id": 1,
  “chatroom": “Chatroom 1”,
  “destination": “backup.zip”,
  “compressionFormat": “zip”,
}
```

- The usual standard methods should be available.



## Re-runnable Jobs

- A re-runnable job can be executed with the custom method `run`.

```
POST /jobs/1:run
```

  - This method returns an LRO making job execution asynchronous.



## Re-runnable Jobs

- Responses and Job persistency
  - It is convenient to decouple persistency policy of generic LRO from the persistence policy applied to Job results.
  - A good way to achieve this is to store LRO resources from job executions in a specific `executions` top level resource.
    - These typically have perpetual persistency
    - Immutable and server-created (no UPDATE or CREATE)

# Re-runnable Jobs

- Interaction with job resource with Execution results.

